

Отчёт по лабораторной работе №5  
Тенигин Валерий 6204-010302D

## Задание 1

Добавил:

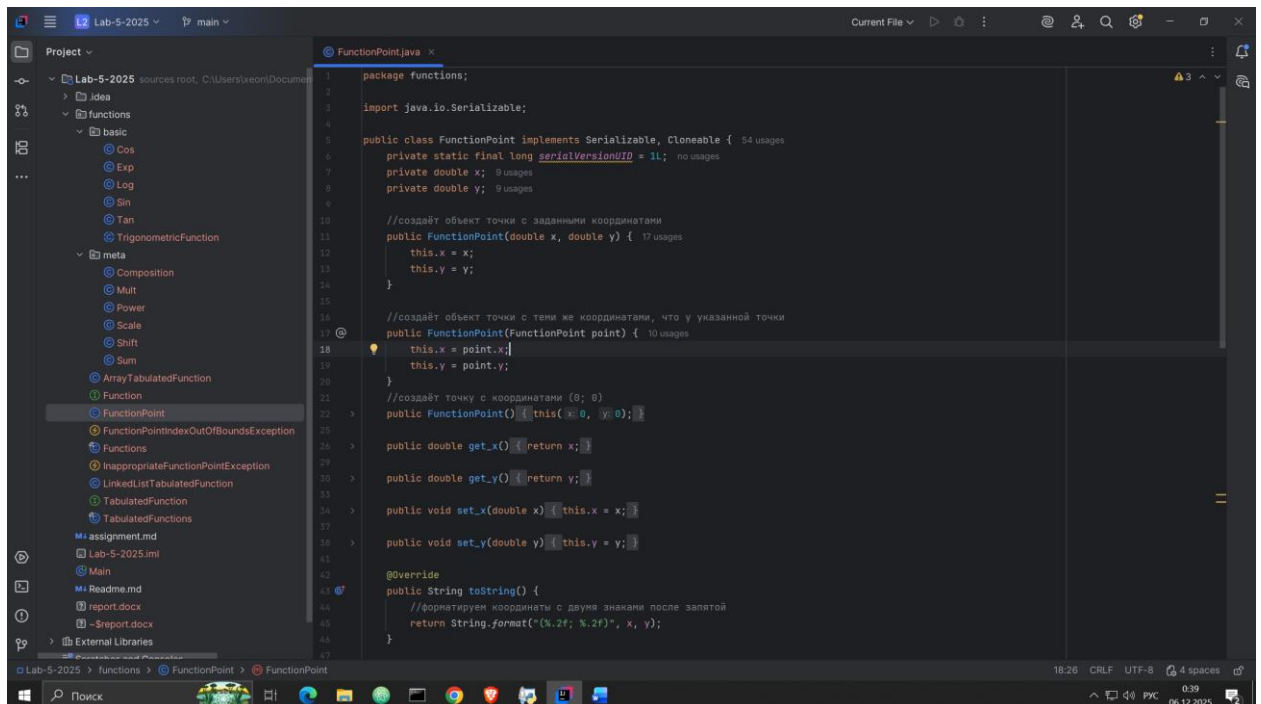
implements Cloneable

toString(): формат (x; y) с точками как разделителями

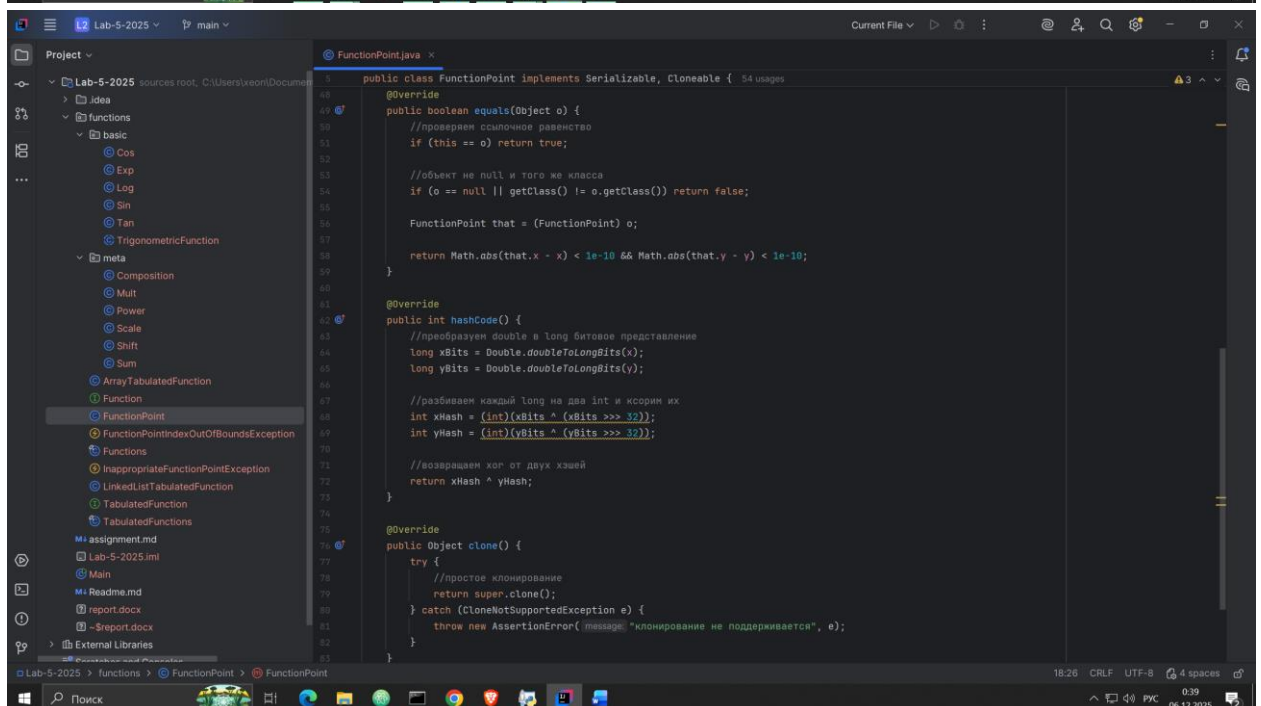
equals(): сравнение с погрешностью

hashCode(): через Double.doubleToLongBits() и xor

clone(): простое клонирование (только примитивные типы)



```
1 package functions;
2
3 import java.io.Serializable;
4
5 public class FunctionPoint implements Serializable, Cloneable { 54 usages
6     private static final long serialVersionUID = 1L;  no usages
7     private double x; 9 usages
8     private double y; 9 usages
9
10    //создаёт объект точки с заданными координатами
11    public FunctionPoint(double x, double y) { 17 usages
12        this.x = x;
13        this.y = y;
14    }
15
16    //создаёт объект точки с теми же координатами, что у указанной точки
17    public FunctionPoint(FunctionPoint point) { 10 usages
18        this.x = point.x;
19        this.y = point.y;
20    }
21
22    //создаёт точку с координатами (0; 0)
23    public FunctionPoint() {this(0, 0);}
24
25    public double get_x() {return x;}
26
27    public double get_y() {return y;}
28
29    public void set_x(double x) {this.x = x;}
30
31    public void set_y(double y) {this.y = y;}
32
33    @Override
34    public String toString() {
35        //форматируем координаты с двумя знаками после запятой
36        return String.format("(%.2f; %.2f)", x, y);
37    }
38
39 }
```



```
40
41
42 @Override
43 public boolean equals(Object o) {
44     //проверяем совпадение равенство
45     if (this == o) return true;
46
47     //объект не null и того же класса
48     if (o == null || getClass() != o.getClass()) return false;
49
50     FunctionPoint that = (FunctionPoint) o;
51
52     return Math.abs(that.x - x) < 1e-10 && Math.abs(that.y - y) < 1e-10;
53 }
54
55 @Override
56 public int hashCode() {
57     //преобразуем double в long битовое представление
58     long xBits = Double.doubleToLongBits(x);
59     long yBits = Double.doubleToLongBits(y);
60
61     //разбиваем каждый long на два int и xorим их
62     int xHash = (int)(xBits ^ (xBits >>> 32));
63     int yHash = (int)(yBits ^ (yBits >>> 32));
64
65     //возвращаем xor от двух хэшей
66     return xHash ^ yHash;
67 }
68
69 @Override
70 public Object clone() {
71     try {
72         //простое клонирование
73         return super.clone();
74     } catch (CloneNotSupportedException e) {
75         throw new AssertionError("клонирование не поддерживается", e);
76     }
77 }
78
79 }
```

## Задание 2

Добавил:

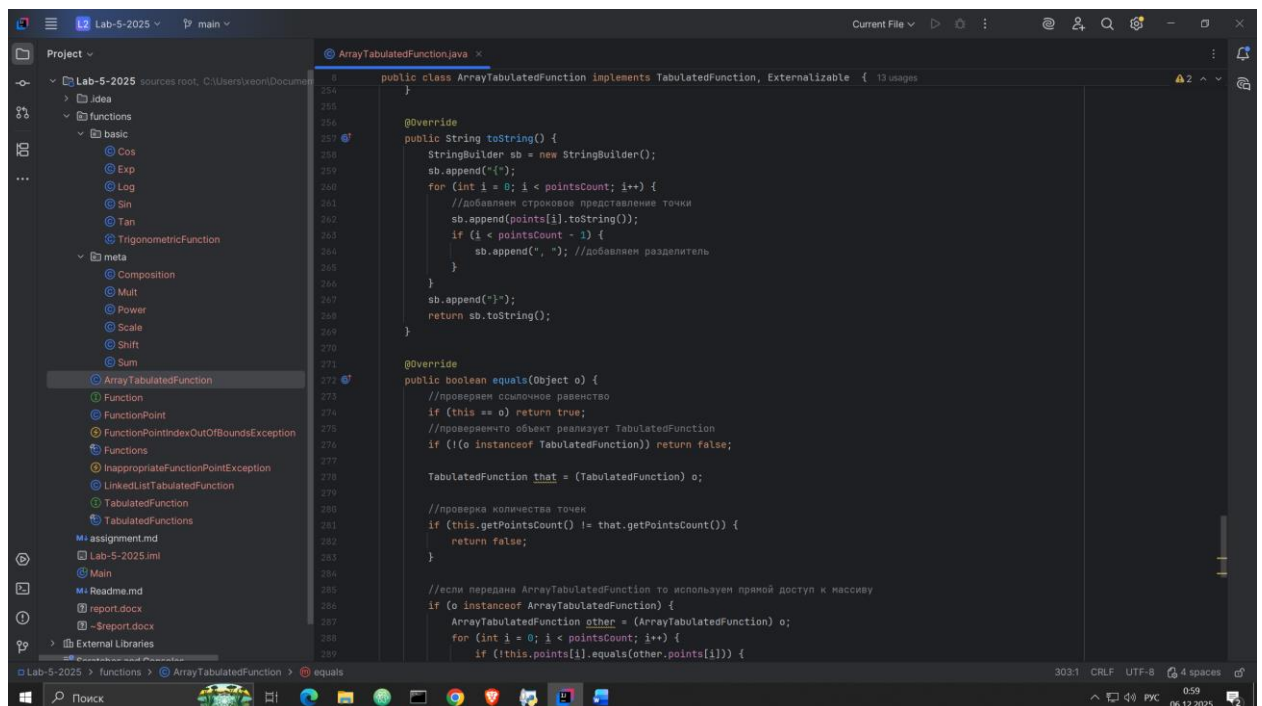
implements Cloneable

toString(): формат {точка1, точка2, ...}

equals(): оптимизация для своего класса и универсальный

hashCode(): включает количество точек + хэши всех точек

clone(): глубокое клонирование массива точек



The screenshot shows the IntelliJ IDEA IDE with the project 'Lab-5-2025' open. The file 'ArrayTabulatedFunction.java' is selected in the Project view on the left. The main editor displays the implementation of the `equals` method, which checks for object equality and point counts. The status bar at the bottom indicates line 303, column 1, with CRLF line endings and 4 spaces.

```
public class ArrayTabulatedFunction implements TabulatedFunction, Externalizable {  
    //...  
    @Override  
    public boolean equals(Object o) {  
        //проверяем ссылочное равенство  
        if (this == o) return true;  
        //проверяем что объект реализует TabulatedFunction  
        if (!(o instanceof TabulatedFunction)) return false;  
  
        TabulatedFunction that = (TabulatedFunction) o;  
  
        //проверка количества точек  
        if (this.getPointsCount() != that.getPointsCount()) {  
            return false;  
        }  
  
        //если передана ArrayTabulatedFunction то используем прямой доступ к массиву  
        if (o instanceof ArrayTabulatedFunction) {  
            ArrayTabulatedFunction other = (ArrayTabulatedFunction) o;  
            for (int i = 0; i < pointsCount; i++) {  
                if (!this.points[i].equals(other.points[i])) {  
                    return false;  
                }  
            }  
        } else {  
            //для любой другой TabulatedFunction используем интерфейсные методы  
            for (int i = 0; i < pointsCount; i++) {  
                FunctionPoint thisPoint = this.getPoint(i);  
                FunctionPoint thatPoint = that.getPoint(i);  
                if (!thisPoint.equals(thatPoint)) {  
                    return false;  
                }  
            }  
        }  
        return true;  
    }  
}
```

This screenshot shows the continuation of the `ArrayTabulatedFunction` class, specifically the `hashCode` and `clone` methods. The `hashCode` method calculates a hash based on the number of points and their individual hashes. The `clone` method performs a deep clone of the points array. The status bar shows line 329, column 1.

```
    @Override  
    public int hashCode() {  
        //включаем количество точек в хэш  
        int result = pointsCount;  
        for (int i = 0; i < pointsCount; i++) {  
            result ^= points[i].hashCode();  
        }  
        return result;  
    }  
  
    @Override  
    public TabulatedFunction clone() {  
        try {  
            ArrayTabulatedFunction cloned = (ArrayTabulatedFunction) super.clone();  
            //глубокое клонирование массива точек  
            cloned.points = new FunctionPoint[points.length];  
            for (int i = 0; i < pointsCount; i++) {  
                cloned.points[i] = (FunctionPoint) points[i].clone();  
            }  
            return cloned;  
        } catch (CloneNotSupportedException e) {  
            throw new AssertionError("клонирование не поддерживается", e);  
        }  
    }  
}
```

This screenshot shows the top of the `ArrayTabulatedFunction` class, including imports for `java.io.*` and `java.io.Serializable`. The class is declared to implement `TabulatedFunction`, `Externalizable`, and `Cloneable`. The status bar shows line 100, column 1.

```
import java.io.*;  
//import java.io.Serializable;  
  
//public class ArrayTabulatedFunction implements TabulatedFunction, Serializable {  
public class ArrayTabulatedFunction implements TabulatedFunction, Externalizable, Cloneable {  
    //private static final long serialVersionUID = 1L;  
  
    private FunctionPoint[] points; //массив типа FunctionPoint 48 usages  
    private int pointsCount; 39 usages  
  
    // Конструктор без параметров для Externalizable  
    public ArrayTabulatedFunction() {} 6 usages  
}
```

## Задание 3

Добавил:

implements Cloneable

toString(): обход связанного списка

equals(): оптимизация для своего класса

hashCode(): обход списка с хог

clone(): пересборка списка с клонированием точек

```
package functions;
import java.io.Serializable;

public class LinkedListTabulatedFunction implements TabulatedFunction, Serializable, Cloneable { 10 usages
    private static final long serialVersionUID = 1L; no usages

    private static class FunctionNode implements Serializable {
        private double x;
        private double y;
        private FunctionNode next;
    }

    private FunctionNode head;
    private int pointsCount;

    public LinkedListTabulatedFunction() {
        head = null;
        pointsCount = 0;
    }

    public void addPoint(double x, double y) {
        FunctionNode node = new FunctionNode();
        node.x = x;
        node.y = y;
        if (head == null) {
            head = node;
        } else {
            FunctionNode current = head;
            while (current.next != null) {
                current = current.next;
            }
            current.next = node;
        }
        pointsCount++;
    }

    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append("{");
        FunctionNode node = head; //начинаем с первой точки
        while (node != null) {
            sb.append(node.point.toString()); //добавляем строковое представление точки
            //если не последняя точка
            if (node.next != null) {
                sb.append(", "); //добавляем разделитель
            }
            node = node.next;
        }
        sb.append("}");
        return sb.toString();
    }

    @Override
    public boolean equals(Object o) {
        //проверим ссылочное равенство
        if (this == o) return true;
        //объект реализует TabulatedFunction
        if (!(o instanceof TabulatedFunction)) return false;

        TabulatedFunction that = (TabulatedFunction) o;
        //проверим количество точек
        if (this.getPointsCount() != that.getPointsCount()) {
            return false;
        }

        //если передана LinkedListTabulatedFunction используем прямой доступ к узлам
        if (o instanceof LinkedListTabulatedFunction) {
            LinkedListTabulatedFunction thatLL = (LinkedListTabulatedFunction) o;
            FunctionNode thisNode = head;
            FunctionNode thatNode = thatLL.head;
            while (thisNode != null) {
                if (!thisNode.equals(thatNode)) {
                    return false;
                }
                thisNode = thisNode.next;
                thatNode = thatNode.next;
            }
            return true;
        }

        //иначе идем по точкам
        for (int i = 0; i < pointsCount; i++) {
            double thisX = this.getX(i);
            double thisY = this.getY(i);
            double thatX = that.getX(i);
            double thatY = that.getY(i);
            if (!thisX.equals(thatX) || !thisY.equals(thatY)) {
                return false;
            }
        }
        return true;
    }

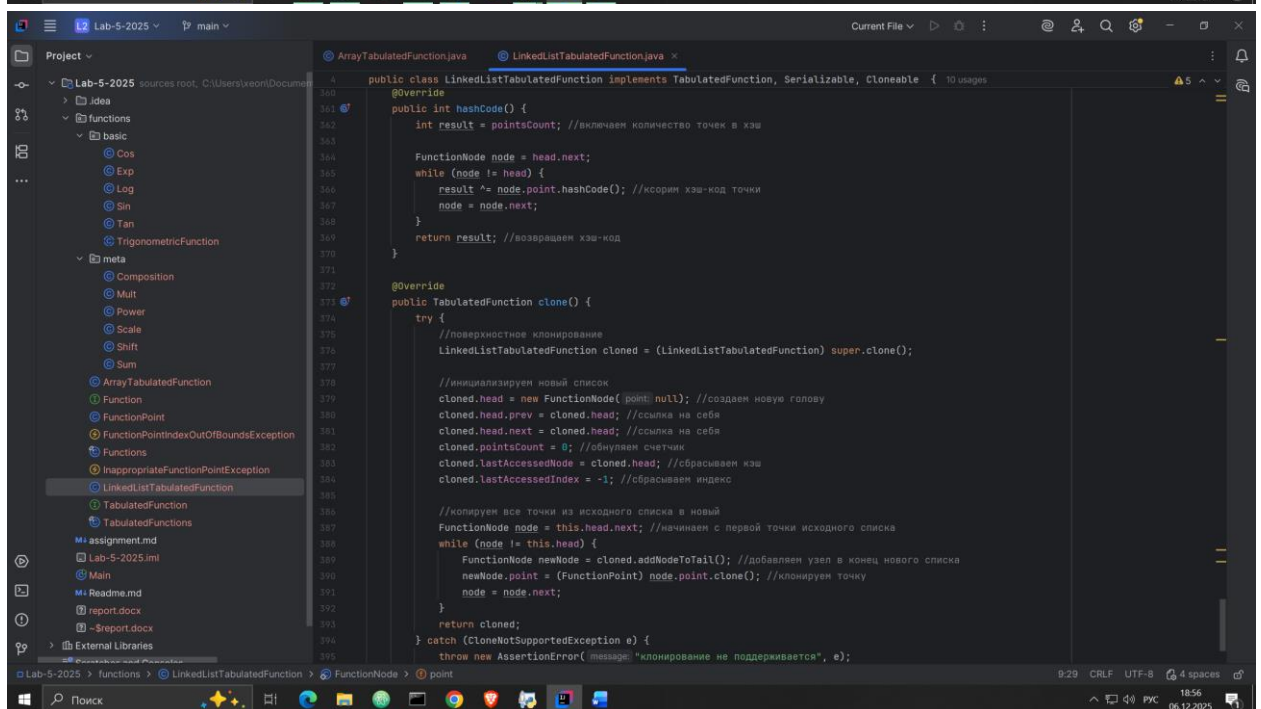
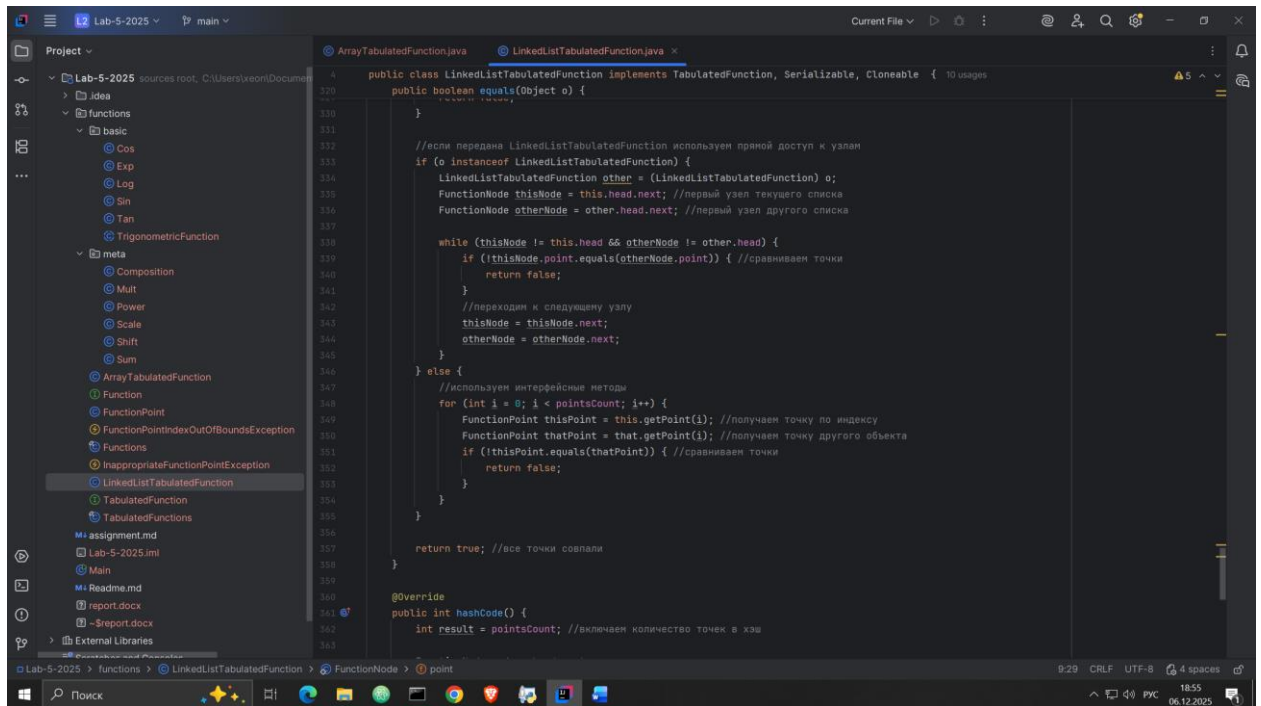
    public double getX(int i) {
        FunctionNode node = head;
        for (int j = 0; j < i; j++) {
            node = node.next;
        }
        return node.x;
    }

    public double getY(int i) {
        FunctionNode node = head;
        for (int j = 0; j < i; j++) {
            node = node.next;
        }
        return node.y;
    }

    public int getPointsCount() {
        return pointsCount;
    }

    public void setPointsCount(int pointsCount) {
        this.pointsCount = pointsCount;
    }

    public void clone() {
        LinkedListTabulatedFunction clone = new LinkedListTabulatedFunction();
        FunctionNode node = head;
        while (node != null) {
            clone.addPoint(node.x, node.y);
            node = node.next;
        }
    }
}
```





#### Задание 4

## Расширил extends Function, Cloneable

## Добавил метод TabulatedFunction clone()

```

1 package functions;
2
3 public interface TabulatedFunction extends Function, Cloneable { 33 usages 2 implementations
4     int getPointsCount(); 31 usages 2 implementations
5     FunctionPoint getPoint(int index); 4 usages 2 implementations
6     void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException; 3 usages 2 implementations
7     double getPointX(int index); 19 usages 2 implementations
8     void setPointX(int index, double x) throws InappropriateFunctionPointException; 2 usages 2 implementations
9     double getPointY(int index); 22 usages 2 implementations
10    void setPointY(int index, double y); 3 usages 2 implementations
11    void deletePoint(int index); 2 usages 2 implementations
12    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException; 3 usages 2 implementations
13
14    TabulatedFunction clone(); 2 implementations
15 }
16

```

## Задание 5

Добавил дополнительные проверки и произвёл тестирование

```
public class Main {  
    public static void main(String[] args) throws InappropriateFunctionPointException {  
  
        System.out.println("\n" + "-".repeat(50));  
        System.out.println("=== ТЕСТЫ ДЛЯ РАБОТОЙНОЙ WS ===");  
        System.out.println("-".repeat(50));  
  
        //(Задание 1)  
        System.out.println("\n=== ТЕСТ 17: FunctionPoint ===");  
        {  
            System.out.println("тестирование FunctionPoint:");  
  
            FunctionPoint p1 = new FunctionPoint( X:1.5, Y:2.5);  
            FunctionPoint p2 = new FunctionPoint( X:1.5, Y:2.5);  
            FunctionPoint p3 = new FunctionPoint( X:1.5000000001, Y:2.5000000001);  
            FunctionPoint p4 = new FunctionPoint( X:1.5, Y:3.0);  
  
            System.out.println("toString():");  
            p1: " + p1.toString();  
            System.out.println("p2: " + p2.toString());  
  
            System.out.println("equals():");  
            System.out.println("p1.equals(p2): " + p1.equals(p2) + " (ожидается true)");  
            System.out.println("p1.equals(p3): " + p1.equals(p3) + " (ожидается true)");  
            System.out.println("p1.equals(p4): " + p1.equals(p4) + " (ожидается false)");  
  
            System.out.println("hashCode():");  
            System.out.println("p1.hashCode(): " + p1.hashCode());  
            System.out.println("p2.hashCode(): " + p2.hashCode());  
            System.out.println("p1.hashCode() == p2.hashCode(): " + (p1.hashCode() == p2.hashCode()));  
  
            System.out.println("clone():");  
            FunctionPoint p1Clone = (FunctionPoint) p1.clone();  
            System.out.println("p1.equals(p1Clone): " + p1.equals(p1Clone));  
            p1 == p1Clone: " + p1 == p1Clone + " (ожидается false)";  
        }  
    }  
}
```

```
=== ТЕСТЫ ДЛЯ РАБОТОЙНОЙ WS ===  
=====
```

```
=== ТЕСТ 17: FunctionPoint ===  
тестирование FunctionPoint:  
toString():  
p1: (1.50; 2.50)  
p2: (1.50; 2.50)  
equals():  
p1.equals(p2): true (ожидается true)  
p1.equals(p3): false (ожидается true)  
p1.equals(p4): false (ожидается false)  
hashCode():  
p1.hashCode(): 2147221504  
p2.hashCode(): 2147221504  
p1.hashCode() == p2.hashCode(): true  
clone():  
p1.equals(p1Clone): true  
p1 == p1Clone: false (ожидается false)
```

```
=== ТЕСТ 18: ArrayTabulatedFunction ===  
тестирование ArrayTabulatedFunction:  
toString():  
arrayFunc1: {(0.00; 0.00), (1.00; 1.00), (2.00; 4.00)}  
arrayFunc2: {(0.00; 0.00), (1.00; 1.00), (2.00; 4.00)}  
equals():  
arrayFunc1.equals(arrayFunc2): true (ожидается true)  
arrayFunc1.equals(arrayFunc3): false (ожидается false)  
arrayFunc1.equals(null): false (ожидается false)  
hashCode():  
arrayFunc1.hashCode(): 1048579  
arrayFunc2.hashCode(): 1048579  
arrayFunc3.hashCode(): 2145386499  
arrayFunc1.hashCode() == arrayFunc2.hashCode(): true
```

```
ТЕСТ ИЗМЕНЕНИЯ ХЭШ-КОДА (незначительные изменения):  
295:18 (3 chars) LF UTF-8 4 spaces 09:30:07
```

```
Project ▾
├── Lab-5-2025
│   ├── source
│   │   ├── functions
│   │   │   ├── basic
│   │   │   │   ├── Cos
│   │   │   │   ├── Exp
│   │   │   │   ├── Log
│   │   │   │   ├── Sin
│   │   │   │   ├── Tan
│   │   │   │   ├── Trigonomet
│   │   │   │   └── meta
│   │   │   │       ├── Composition
│   │   │   │       ├── Mult
│   │   │   │       ├── Power
│   │   │   │       ├── Scale
│   │   │   │       ├── Shift
│   │   │   │       ├── Sum
│   │   │   │       ├── ArrayTabulated
│   │   │   │       ├── Function
│   │   │   │       ├── FunctionPoint
│   │   │   │       ├── FunctionPoint1
│   │   │   │       ├── Functions
│   │   │   │       ├── InappropriateF
│   │   │   │       ├── LinkedListTabu
│   │   │   │       ├── TabulatedFunc
│   │   │   │       └── TabulatedFunc
│   │   │   └── out
│   │   │       ├── assignment.md
│   │   │       ├── externalizable.dat
│   │   │       ├── Lab-5-2025.iml
│   │   │       ├── Main
│   │   │       ├── Readme.md
│   │   │       ├── report.docx
│   │   │       └── report.docx
│   │   └── out
│   │       ├── assignment.md
│   │       ├── externalizable.dat
│   │       ├── Lab-5-2025.iml
│   │       ├── Main
│   │       ├── Readme.md
│   │       ├── report.docx
│   │       └── report.docx
│   └── out
│       ├── assignment.md
│       ├── externalizable.dat
│       ├── Lab-5-2025.iml
│       ├── Main
│       ├── Readme.md
│       ├── report.docx
│       └── report.docx
└── out
    ├── assignment.md
    ├── externalizable.dat
    ├── Lab-5-2025.iml
    ├── Main
    ├── Readme.md
    ├── report.docx
    └── report.docx

Main.java
13 public class Main {
14     public static void main(String[] args) throws InappropriateFunctionPointException {
15         System.out.println("n== TECT 18: ArrayTabulatedFunction ==");
16         System.out.println("тестирование ArrayTabulatedFunction:");
17         FunctionPoint[] arrayPoints1 = {
18             new FunctionPoint(0.0, 0.0),
19             new FunctionPoint(1.0, 1.0),
20             new FunctionPoint(2.0, 4.0)
21         };
22         FunctionPoint[] arrayPoints2 = {
23             new FunctionPoint(0.0, 0.0),
24             new FunctionPoint(1.0, 1.0),
25             new FunctionPoint(2.0, 4.0)
26         };
27         FunctionPoint[] arrayPoints3 = {
28             new FunctionPoint(0.0, 0.0),
29             new FunctionPoint(1.0, 1.0),
30             new FunctionPoint(2.0, 4.0)
31         };
32         ArrayTabulatedFunction arrayFunc1 = new ArrayTabulatedFunction(arrayPoints1);
33         ArrayTabulatedFunction arrayFunc2 = new ArrayTabulatedFunction(arrayPoints2);
34         ArrayTabulatedFunction arrayFunc3 = new ArrayTabulatedFunction(arrayPoints3);
35         System.out.println("toString():");
36         System.out.println("arrayFunc1: " + arrayFunc1.toString());
37         System.out.println("arrayFunc2: " + arrayFunc2.toString());
38         System.out.println("equals():");
39         System.out.println("arrayFunc1.equals(arrayFunc2): " + arrayFunc1.equals(arrayFunc2) + " (ожидается true)");
40         System.out.println("arrayFunc1.equals(arrayFunc3): " + arrayFunc1.equals(arrayFunc3) + " (ожидается true)");
41         System.out.println("arrayFunc1.equals(null): " + arrayFunc1.equals(null) + " (ожидается false)");
42     }
43 }

Run
Main
arrayFunc1.equals(arrayFunc2): true
p1 == p1Clone: false (ожидается false)

=== TECT 18: ArrayTabulatedFunction ===
тестирование ArrayTabulatedFunction:
toString():
arrayFunc1: {(0,0; 0,0), (1,0; 1,0), (2,0; 4,0)}
arrayFunc2: {(0,0; 0,0), (1,0; 1,0), (2,0; 4,0)}
equals():
arrayFunc1.equals(arrayFunc2): true (ожидается true)
arrayFunc1.equals(arrayFunc3): false (ожидается false)
arrayFunc1.equals(null): false (ожидается false)
hashCode():
arrayFunc1.hashCode(): 1048579
arrayFunc2.hashCode(): 1048579
arrayFunc3.hashCode(): 2145386499
arrayFunc1.hashCode() == arrayFunc2.hashCode(): true
тест изменения хэш-кода (незначительное изменение):
исходный хэш: 1048579
после изменения arrayFunc1.setPointY(1, 1.001):
новый хэш: -1822115727
хэш изменился: true (ожидается true)
clone():
arrayFunc1: {(0,0; 0,0), (1,0; 1,0), (2,0; 4,0)}
arrayFunc1Clone: {(0,0; 0,0), (1,0; 1,0), (2,0; 4,0)}
arrayFunc1.equals(arrayFunc1Clone): true (ожидается true)
arrayFunc1 == arrayFunc1Clone: false (ожидается false)
проверка глубокого клонирования:
после изменения arrayFunc1Clone.setPointY(1, 999.0):
arrayFunc1.getPointY(1): 1.001
arrayFunc1Clone.getPointY(1): 999.0
разные значения: true (ожидается true)
объекты-клоны не изменились: true (ожидается true)

=== TECT 19: LinkedListTabulatedFunction ===
тестирование LinkedListTabulatedFunction:
295:16 (3 chars) LF UTF-8 4 spaces 1956 06.12.2025
```

```
Project ▾
├── Lab-5-2025
│   ├── source
│   │   ├── functions
│   │   │   ├── basic
│   │   │   │   ├── Cos
│   │   │   │   ├── Exp
│   │   │   │   ├── Log
│   │   │   │   ├── Sin
│   │   │   │   ├── Tan
│   │   │   │   ├── Trigonomet
│   │   │   │   └── meta
│   │   │   │       ├── Composition
│   │   │   │       ├── Mult
│   │   │   │       ├── Power
│   │   │   │       ├── Scale
│   │   │   │       ├── Shift
│   │   │   │       ├── Sum
│   │   │   │       ├── ArrayTabulated
│   │   │   │       ├── Function
│   │   │   │       ├── FunctionPoint
│   │   │   │       ├── FunctionPoint1
│   │   │   │       ├── Functions
│   │   │   │       ├── InappropriateF
│   │   │   │       ├── LinkedListTabu
│   │   │   │       ├── TabulatedFunc
│   │   │   │       └── TabulatedFunc
│   │   │   └── out
│   │   │       ├── assignment.md
│   │   │       ├── externalizable.dat
│   │   │       ├── Lab-5-2025.iml
│   │   │       ├── Main
│   │   │       ├── Readme.md
│   │   │       ├── report.docx
│   │   │       └── report.docx
│   │   └── out
│   │       ├── assignment.md
│   │       ├── externalizable.dat
│   │       ├── Lab-5-2025.iml
│   │       ├── Main
│   │       ├── Readme.md
│   │       ├── report.docx
│   │       └── report.docx
│   └── out
│       ├── assignment.md
│       ├── externalizable.dat
│       ├── Lab-5-2025.iml
│       ├── Main
│       ├── Readme.md
│       ├── report.docx
│       └── report.docx
└── out
    ├── assignment.md
    ├── externalizable.dat
    ├── Lab-5-2025.iml
    ├── Main
    ├── Readme.md
    ├── report.docx
    └── report.docx

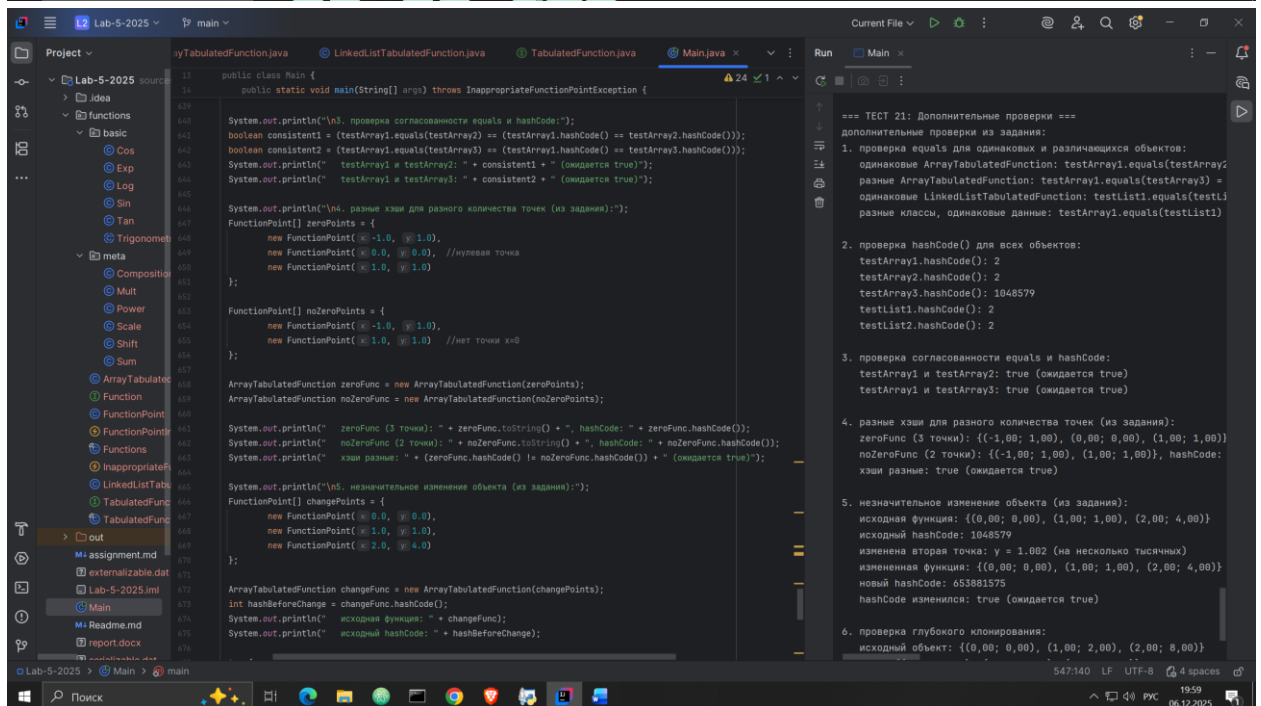
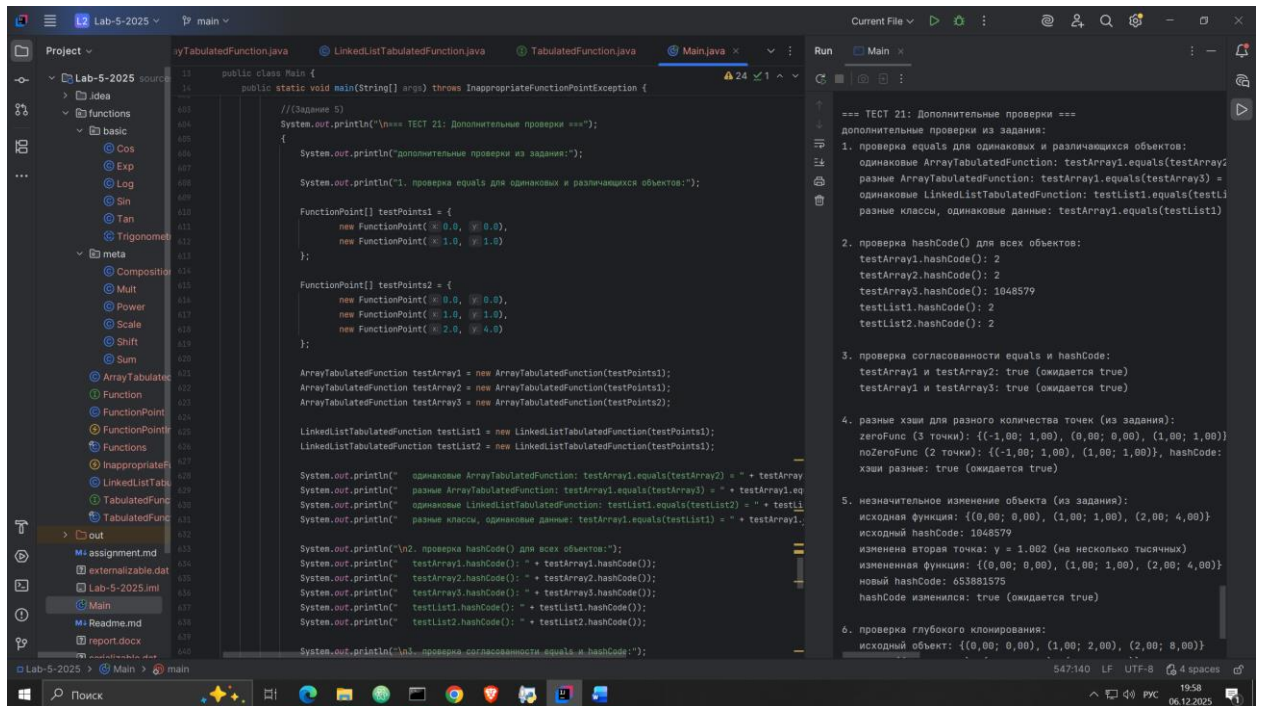
Main.java
13 public class Main {
14     public static void main(String[] args) throws InappropriateFunctionPointException {
15         System.out.println("arrayFunc1.hashCode(): " + arrayFunc1.hashCode());
16         System.out.println("arrayFunc2.hashCode(): " + arrayFunc2.hashCode());
17         System.out.println("arrayFunc3.hashCode(): " + arrayFunc3.hashCode());
18         System.out.println("arrayFunc1.hashCode() == arrayFunc2.hashCode(): " + (arrayFunc1.hashCode() == arrayFunc2.hashCode()) + " (ожидается true)");
19         System.out.println("тест изменения хэш-кода (незначительное изменение):");
20         int originalHash = arrayFunc1.hashCode();
21         System.out.println("исходный хэш: " + originalHash);
22         try {
23             arrayFunc1.setPointY(1, 1.001); //незначительное изменение
24             int newHash = arrayFunc1.hashCode();
25             System.out.println("после изменения arrayFunc1.setPointY(1, 1.001):");
26             System.out.println("новый хэш: " + newHash);
27             System.out.println("хэш изменился: " + (originalHash != newHash) + " (ожидается true)");
28         } catch (Exception e) {
29             System.out.println("ошибка при изменении: " + e.getMessage());
30         }
31         System.out.println("clone():");
32         ArrayTabulatedFunction arrayFunc1Clone = (ArrayTabulatedFunction) arrayFunc1.clone();
33         System.out.println("arrayFunc1: " + arrayFunc1);
34         System.out.println("arrayFunc1Clone: " + arrayFunc1Clone);
35         System.out.println("arrayFunc1.equals(arrayFunc1Clone): " + arrayFunc1.equals(arrayFunc1Clone));
36         System.out.println("arrayFunc1 == arrayFunc1Clone: " + (arrayFunc1 == arrayFunc1Clone) + " (ожидается false)");
37         System.out.println("проверка глубокого клонирования:");
38         try {
39             arrayFunc1Clone.setPointY(1, 999.0); //изменил клон
40             System.out.println("после изменения arrayFunc1Clone.setPointY(1, 999.0):");
41             System.out.println("arrayFunc1.getPointY(1): " + arrayFunc1.getPointY(1));
42             System.out.println("arrayFunc1Clone.getPointY(1): " + arrayFunc1Clone.getPointY(1));
43             System.out.println("arrayFunc1Clone.getPointY(1) != arrayFunc1.getPointY(1): " + (arrayFunc1Clone.getPointY(1) != arrayFunc1.getPointY(1)) + " (ожидается true)");
44             System.out.println("разные значения: " + (arrayFunc1.getPointY(1) != arrayFunc1Clone.getPointY(1)) + " (ожидается true)");
45             System.out.println("объекты-клоны не изменились: " + (arrayFunc1.equals(arrayFunc1Clone) == true) + " (ожидается true)");
46         } catch (Exception e) {
47             System.out.println("ошибка при проверке глубокого клонирования: " + e.getMessage());
48         }
49     }
50 }

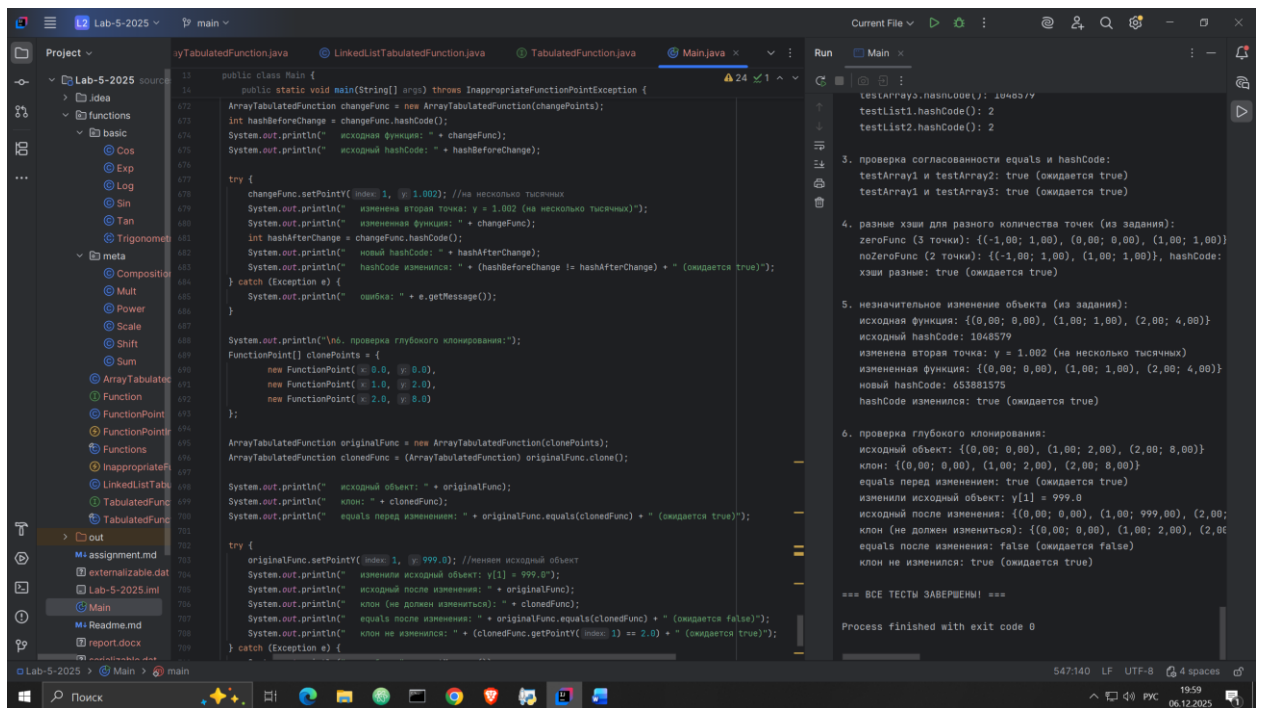
Run
Main
arrayFunc1.hashCode(): 1048579
arrayFunc2.hashCode(): 1048579
arrayFunc3.hashCode(): 2145386499
arrayFunc1.hashCode() == arrayFunc2.hashCode(): true
тест изменения хэш-кода (незначительное изменение):
исходный хэш: 1048579
после изменения arrayFunc1.setPointY(1, 1.001):
новый хэш: -1822115727
хэш изменился: true (ожидается true)
clone():
arrayFunc1: {(0,0; 0,0), (1,0; 1,0), (2,0; 4,0)}
arrayFunc1Clone: {(0,0; 0,0), (1,0; 1,0), (2,0; 4,0)}
arrayFunc1.equals(arrayFunc1Clone): true (ожидается true)
arrayFunc1 == arrayFunc1Clone: false (ожидается false)
проверка глубокого клонирования:
после изменения arrayFunc1Clone.setPointY(1, 999.0):
arrayFunc1.getPointY(1): 1.001
arrayFunc1Clone.getPointY(1): 999.0
разные значения: true (ожидается true)
объекты-клоны не изменились: true (ожидается true)

=== TECT 19: LinkedListTabulatedFunction ===
тестирование LinkedListTabulatedFunction:
295:16 (3 chars) LF UTF-8 4 spaces 1956 06.12.2025
```









Во время тестов была найдена одна ошибка (p1.equals(p3): false (ожидается true))

```

=== ТЕСТЫ ДЛЯ ЛАБОРАТОРНОЙ №5 ===
=====

=== ТЕСТ 17: FunctionPoint ===
тестирование FunctionPoint:
toString():
p1: (1,50; 2,50)
p2: (1,50; 2,50)
equals():
p1.equals(p2): true (ожидается true)
p1.equals(p3): false (ожидается true)
p1.equals(p4): false (ожидается false)
hashCode():
p1.hashCode(): 2147221504

```

Ошибка убрана путём увеличения погрешности. Вместо  $1e-10$  стало  $1e-9$

```
@Override
public boolean equals(Object o) {
    //проверяем ссылочное равенство
    if (this == o) return true;

    //объект не null и того же класса
    if (o == null || getClass() != o.getClass()) return false;

    FunctionPoint that = (FunctionPoint) o;

    return Math.abs(that.x - x) < 1e-9 && Math.abs(that.y - y) < 1e-9;
}
```

```
=== ТЕСТ 17: FunctionPoint ===
тестирование FunctionPoint:
toString():
    p1: (1,50; 2,50)
    p2: (1,50; 2,50)
equals():
    p1.equals(p2): true (ожидается true)
    p1.equals(p3): true (ожидается true)
    p1.equals(p4): false (ожидается false)
hashCode():
    p1.hashCode(): 2147221504
    p2.hashCode(): 2147221504
    p1.hashCode() == p2.hashCode(): true
clone():
    p1.equals(p1Clone): true
    p1 == p1Clone: false (ожидается false)
```

```
=== ТЕСТ 18: ArrayTabulatedFunction ===
тестирование ArrayTabulatedFunction:
```

429:1 LF UTF-8 4 spaces

^ 🖥️ 🔊 РУС

20:13  
06.12.2025